gRNA Assignment Results

2026-01-10

Outline

1. Overview of what we are benchmarking

a. summary of methods

b. summary of datasets

2. Results

a. Simulated data: ground truth analysis, and computational results

b. Real data: comparison between methods, and computational results

1. Overview of what we are benchmarking

1a. Methods

For this part of the project, we are assessing five methods (two of which are versions of sceptre) on two real datasets and one simulated dataset.

a. `pertpy`'s `assign_mixture_model()`

- They fit a 2-component mixture to the non-zero log2-transformed counts, with the (log2) counts for the non-perturbed cells being modeled as Pois(λ) and the (log2) perturbed counts are modeled as $\mathcal{N}(\mu, \sigma)$. The mixing proportions are $\pi = (\pi_0, \pi_1)$, for 4 parameters in total
- They do a "separation penalty" where they add 10 to the log posterior when $\lambda < \mu$, to encourage the Poisson component to stay left of the Gaussian and to avoid component flipping
- Cells with 0 UMI counts for a given guide are always assigned to not express that guide. Guides with 1 non-zero cell are skipped.
- There is no normalizing of any sort, and no information is shared between the assignment steps for each guide. No cell-level statistics are used for normalization, or anything like that
- Low and high MOI are handled the exact same
- The model is fit using NUTS MCMC via NumPyro
- Priors: $\mu \sim \mathcal{N}(\mu_0, \sigma_0)$ with a default of $(\mu_0, \sigma_0) = (3, 2)$; $\sigma \sim \text{HalfNormal}(\tau_0)$ with a default of $\tau_0 = 1$; $\lambda \sim \text{Exp}(r)$ with a default of $r = 0.2$; and $\pi \sim \text{Dirichlet}(\alpha_0, \alpha_1)$ with a default of $\alpha_1 = 0.15$.
- By default, 50 warmup samples are done and then 100 samples are kept
- Hard-coded is that each guide starts with the same seed. This may have weird consequences

b. `CLEANSER`

- There are two models here: CROP-seq (`--cs`) and Direct Capture (`--dc`)
- CROP-seq: ambient UMI counts modeled as Pois(λ), perturbed counts as NegBin(μ_1, θ_1) (mean and overdispersion, respectively). Parameter bounds: $\lambda \in [10^{-6}, 5]$; $\mu_1 \geq \lambda$; and $\theta_1 \geq 10^{-6}$

- Direct Capture: ambient model is NegBin(μ_0, θ_0), and perturbed model is NegBin(μ_1, θ_1). Parameter bounds: $\mu_0 \geq 10^{-6}$; $\theta_0 \geq 0.1$; $\mu_1 \geq 5$; $\theta_1 \geq 0.1$.
- In both cases, the mixing proportion is $r \in [10^{-6}, 10^{-1}]$
- Each guide only uses cells with non-zero counts. A cell with all zero counts will be completely excluded. They do a "zero truncation correction" to remove the bias introduced in the parameter estimates due to only fitting on the positive counts.
- Normalization: each cell c gets a factor L_c defined as (cell library size) / (average library size). L_c is used to scale each cell's perturbed component mean parameter, so e.g. the mean parameter for cell c in CROP-seq would be $L_c\mu_1$, not just μ_1. For DC, the ambient mean parameter is also scaled by L_c. This is the only information shared between guides. One complexity: only cell UMI counts at or below the `--lpf` value are counted towards the cell library size. By default `lpf = 2`, so really this is like the ambient library size. The idea is to exclude the perturbed UMI counts from the library size. Setting `lpf = 0` means no filtering is done. [Note: in the fishash paper, they stick to the default `lpf` everywhere as we do too]. If a cell's library size is computed as 0 due to the `lpf` filtering, they set it to 1.
- They directly model the raw counts, aside from the filtering and normalization described above
- No MOI settings
- The model is fit via Stan, with the following parameters: in both cases $r \sim \text{Beta}(1, 10)$.
- For CS, $\lambda \sim \text{LogNormal}(\log(1.1), \log(1.1))$; $\mu_1 \sim \text{LogNormal}(\log(100), \log(100))$; and $\theta_1 \sim \text{Beta}(1, 10)$
- For DC, $\mu_0 \sim \text{LogNormal}(\log(1.075), \log(1.1))$ and $\theta_0 \sim \text{LogNormal}(\log(1.1), \log(1.1))$; and $\mu_1 \sim \text{LogNormal}(\log(100), \log(3))$ and $\theta_1 \sim \text{LogNormal}(\log(3), \log(2))$
- Other MCMC parameters: 4 chains by default; 300 warmup iterations and then 1000 samples kept per chain; HMC/NUTS via Stan
- Each guide runs on a unique seed and is a totally independent fitting, aside from the L_c
- The outputted posterior probabilities for each guide-cell pair is the median over all MCMC samples (so over all 4000 samples by default)

c. `crispat`'s `ga_poisson_gauss()`

- They model the log2 ambient counts as Pois(λ) (technically a continuous Poisson) with a prior of $\lambda \sim \text{LogNormal}(0, 1)$, and the log2 perturbed counts as $\mathcal{N}(\mu, \sigma)$ with priors of $\mu \sim \mathcal{N}(3, 2)$ and $\sigma \sim \text{LogNormal}(2, 1)$. The mixing weights are $\pi = (\pi_0, \pi_1) \sim \text{Dirichlet}(0.9, 0.1)$.
- Only the non-zero raw counts are considered for each guide. UMIs of 0 do not get used. Guides with only 1 non-zero cell or where the max UMI count is at most 1 are skipped.
- Each guide is fit totally independently. No information is shared
- There are no MOI settings
- The model is fit using Stochastic Variational Inference via Pyro. Then they do MAP estimation, and assign a cell as perturbed if P(Gaussian component | data) > 0.5.

d. `SCEPTRE`

For `SCEPTRE`, we have two different implementations of the same underlying algorithm:

1. The `sceptre` R package
2. The `sceptre-pipeline` nextflow pipeline

TODO: fill this in

1b. Datasets

Currently we are benchmarking on the following datasets:

1. Simulated
2. Gasperini (high MOI)
3. Replogle-RD7 (low MOI)

TODO: add details on real datasets

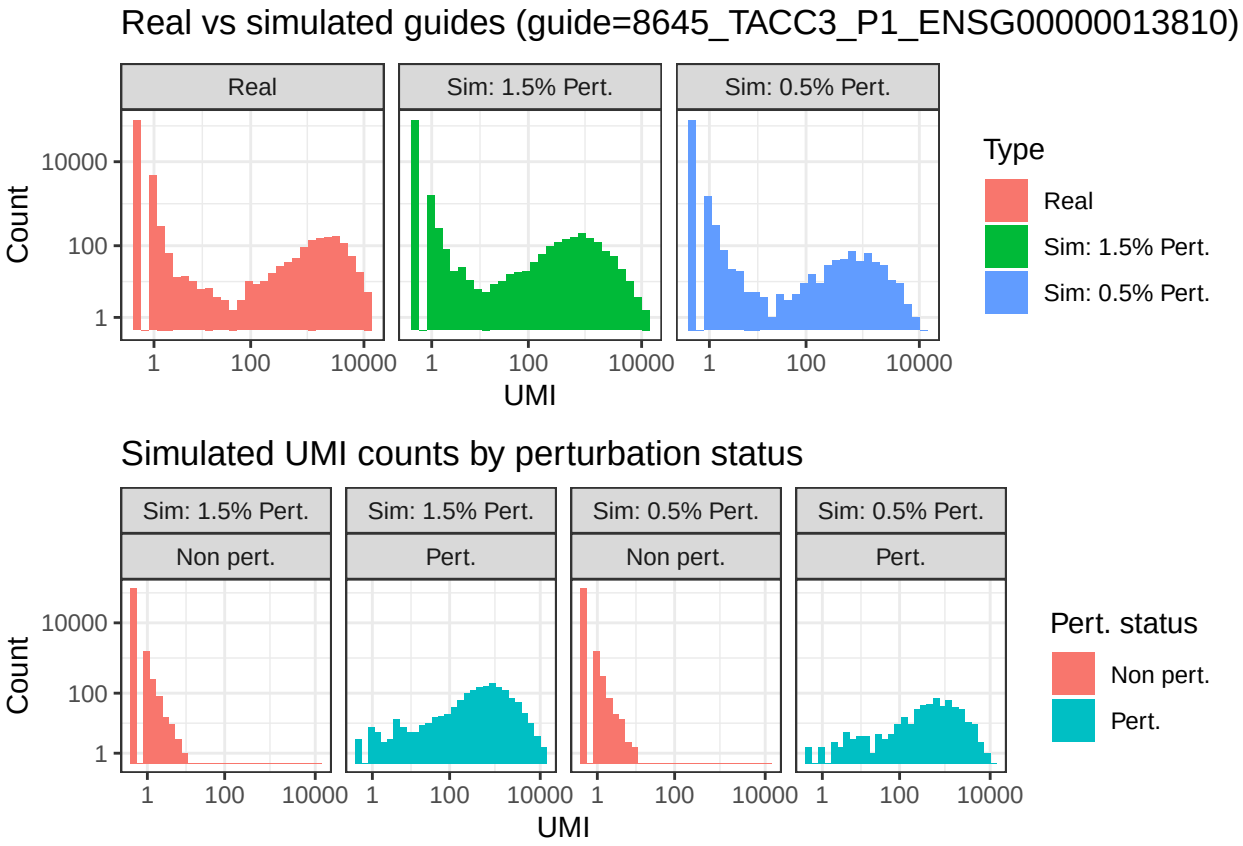
1b(i): The simulated datasets

- **Assignment step:** Run `sceptre-pipeline` on Replogle-rd7 with all default settings to get guide assignments for this dataset using the mixture method.
- **Guide choice:** We only consider the guide `8645_TACC3_P1_ENSG00000013810`. This guide was chosen by first restricting to the guides between the 95% and 99% quantiles for total number of expressed cells, as determined by `sceptre-pipeline`, and then picking the guide from among those with the greatest number of non-zero cells. The idea behind this was to find a highly expressed guide that we could downsample and still have realistic results for.
- **Model fitting:** Fit two models on the real data using cell covariates $\log(\text{grna\_n\_nonzero} + 1)$ and $\log(\text{grna\_n\_umis} + 1)$:
- Negative binomial GLM for UMI counts: umi_counts $\sim$ covariates
- Logistic regression for perturbation probability: sceptre_assignments $\sim$ covariates
- **Cell subsampling:** Subsample to 100,000 cells, keeping all originally perturbed cells plus randomly sampled non-perturbed cells
- **Simulation procedure:** For each simulated guide:
- Generate perturbation indicators via Bernoulli(expit($\hat{\eta}_{\text{pert}}+\beta_0$)), where $\hat{\eta}_{\text{pert}}$ is the fitted logit probability and β_0 is an intercept adjustment. β_0 will be varied to change the overall perturbation rate.
- Generate UMI counts from a two-component negative binomial mixture:

$$Y_i \mid Z_i \sim \text{NegBin}(\mu_i, \theta_i)$$

$$\log(\mu_i) = \hat{\eta}_{\text{umi},i} + \beta_{\text{pert}} \cdot Z_i + \beta_{\text{non-pert}} \cdot (1 - Z_i)$$

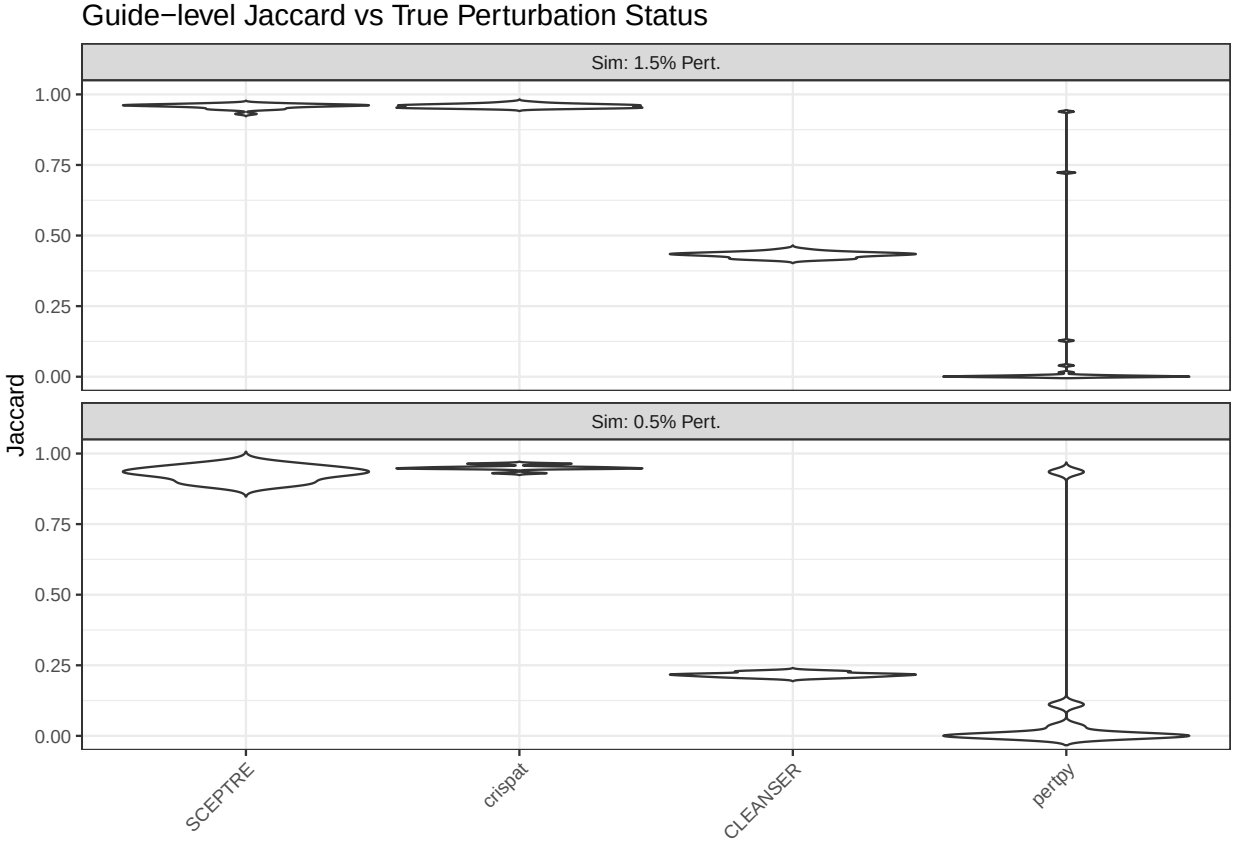
  where Z_i is the perturbation indicator, $\hat{\eta}_{\text{umi},i}$ is the fitted log-mean, $\theta_i = \theta_{\text{pert}}$ if perturbed or $\theta_{\text{non-pert}}$ otherwise
- Clip simulated UMI counts to the maximum observed value in real data
- **Run 1 (replogle-rd7_simulated_100k_0.015-pert):**
- 25 simulated guides
- Parameters: $\beta_0 = 2$, $\beta_{\text{non-pert}} = -5$, $\beta_{\text{pert}} = 5.25$, $\theta_{\text{pert}} = 10$, $\theta_{\text{non-pert}} = 0.1$
- Results in approximately 1.5% perturbation rate
- **Run 2 (replogle-rd7_simulated_100k_0.005-pert):**
- 10 simulated guides
- Parameters: $\beta_0 = 1$, $\beta_{\text{non-pert}} = -5$, $\beta_{\text{pert}} = 5.25$, $\theta_{\text{pert}} = 10$, $\theta_{\text{non-pert}} = 0.1$
- Results in approximately 0.5% perturbation rate

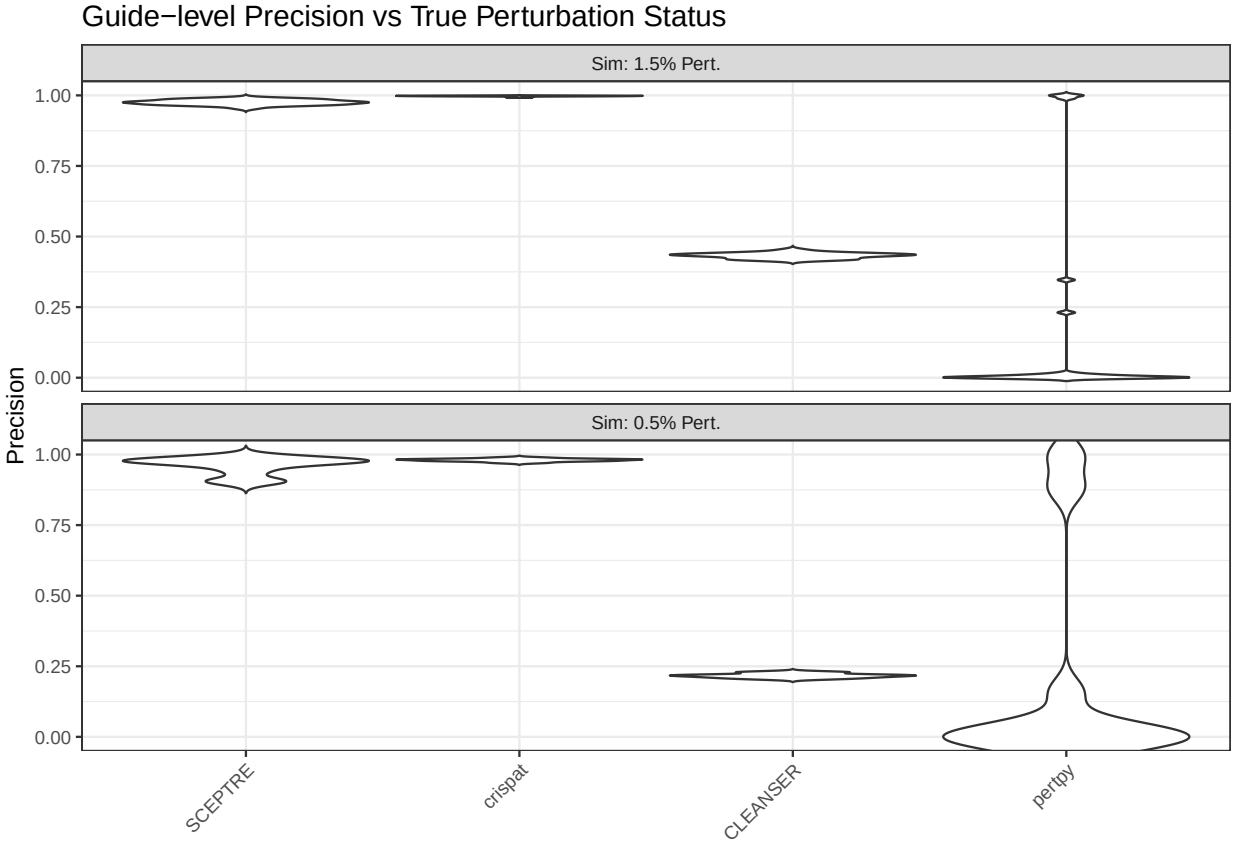

Real vs simulated guides (guide=8645_TACC3_P1_ENSG00000013810)
Real
Sim: 1.5% Pert.
Sim: 0.5% Pert.
Count
10000
100
1
1
100
10000
UMI
Type
Real
Sim: 1.5% Pert.
Sim: 0.5% Pert.

Simulated UMI counts by perturbation status
Sim: 1.5% Pert.
Non pert.
Sim: 1.5% Pert.
Pert.
Sim: 0.5% Pert.
Non pert.
Sim: 0.5% Pert.
Pert.
Count
10000
100
1
1
100
10000
UMI
Pert. status
Non pert.
Pert.

2. Results

2a. Results on simulated data





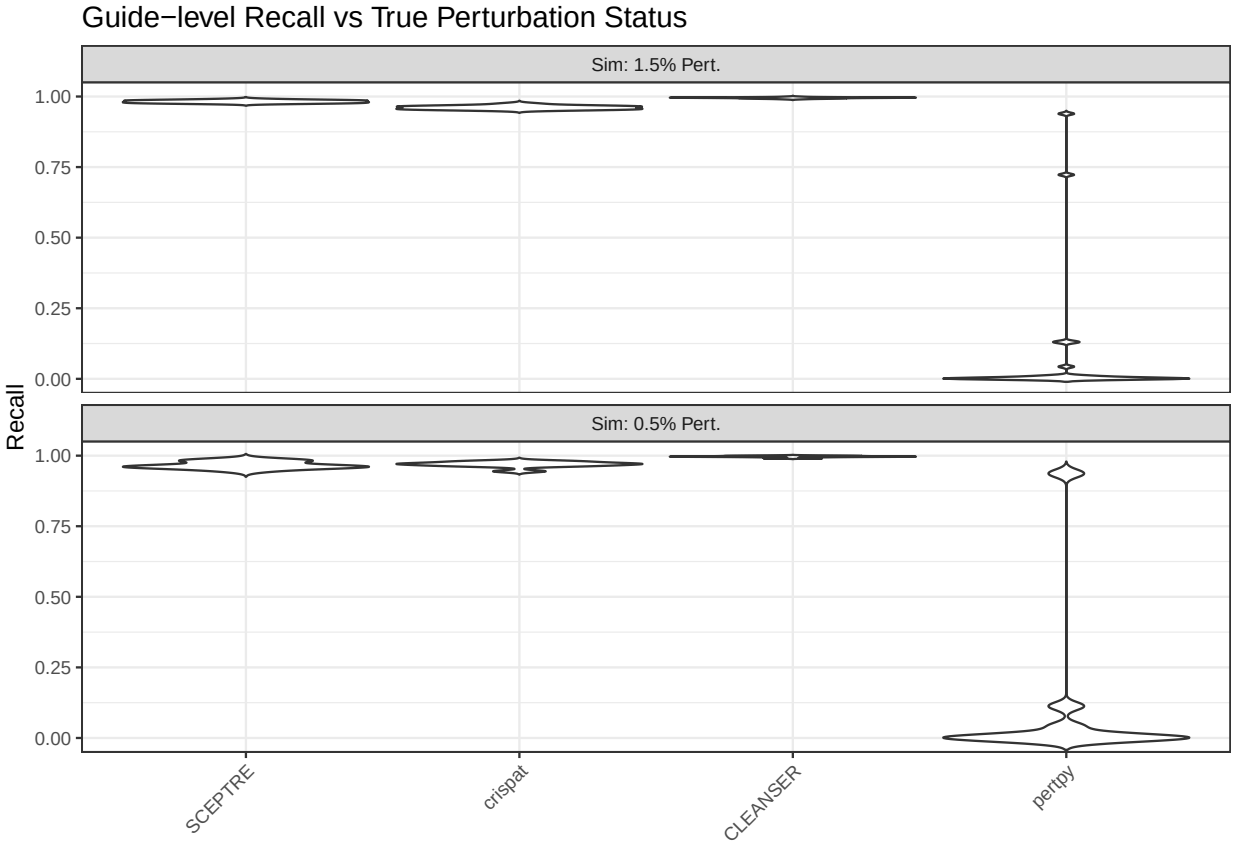


Table 1: Mean (2.5%, 97.5%) over guides

dataset	method	Jaccard	Precision	Recall	Num. Assn. Cells
1.5% Pert	SCEPTRE	0.958 (0.940, 0.969)	0.975 (0.959, 0.989)	0.982 (0.975, 0.990)	1467 (1400, 1544)
1.5% Pert	crispat	**0.959 (0.950, 0.971)**	**0.998 (0.994, 0.999)**	0.961 (0.952, 0.974)	1403 (1357, 1469)
1.5% Pert	pertpy	0.075 (0.000, 0.809)	0.260 (0.002, 1.000)	0.081 (0.000, 0.809)	1013 (0, 5544)
1.5% Pert	CLEANSER	0.431 (0.416, 0.448)	0.432 (0.417, 0.449)	**0.995 (0.991, 0.998)**	3354 (3230, 3447)
0.5% Pert	SCEPTRE	0.928 (0.889, 0.964)	0.960 (0.904, 0.990)	0.966 (0.949, 0.984)	538 (488, 596)
0.5% Pert	crispat	**0.950 (0.934, 0.964)**	**0.980 (0.972, 0.988)**	0.969 (0.949, 0.981)	528 (497, 563)
0.5% Pert	pertpy	0.108 (0.000, 0.750)	0.510 (0.014, 0.989)	0.110 (0.000, 0.751)	258 (0, 1537)
0.5% Pert	CLEANSER	0.217 (0.205, 0.229)	0.217 (0.205, 0.230)	**0.997 (0.991, 1.000)**	2454 (2401, 2531)

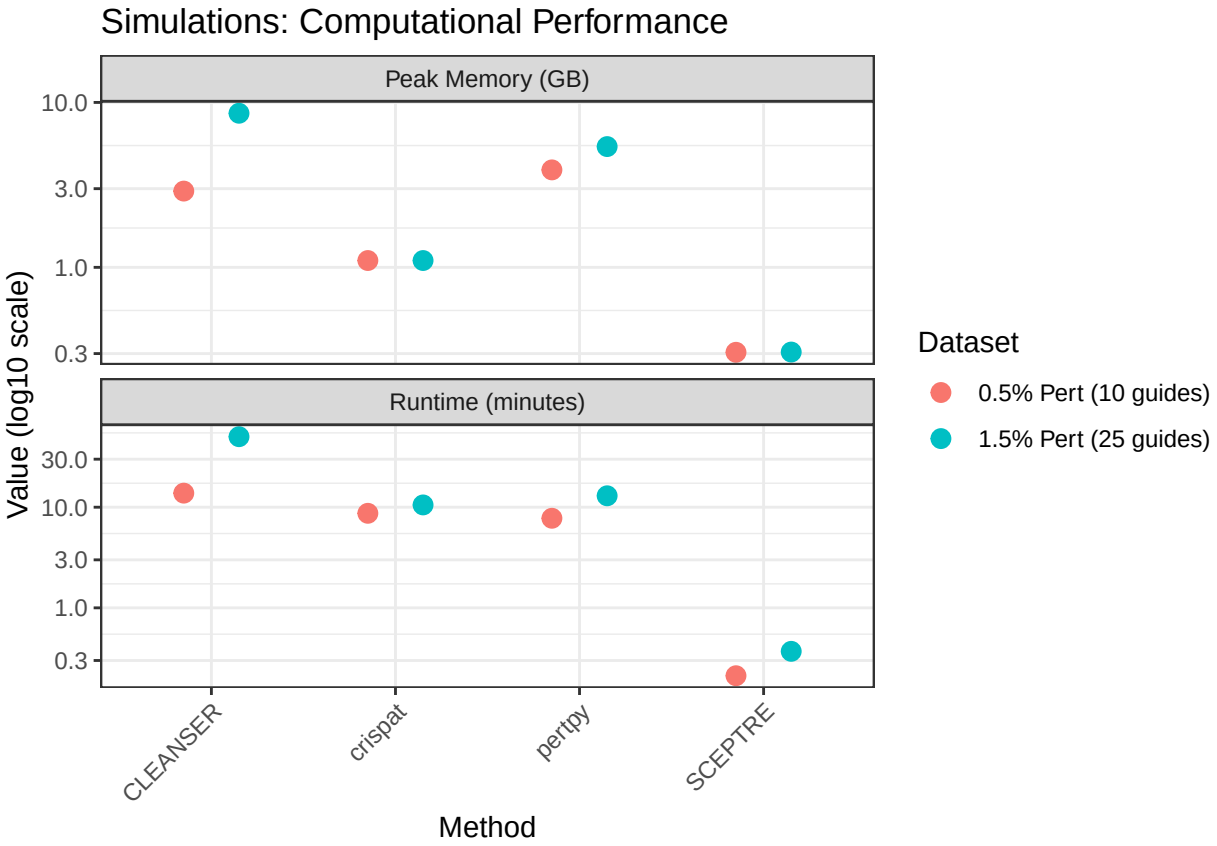


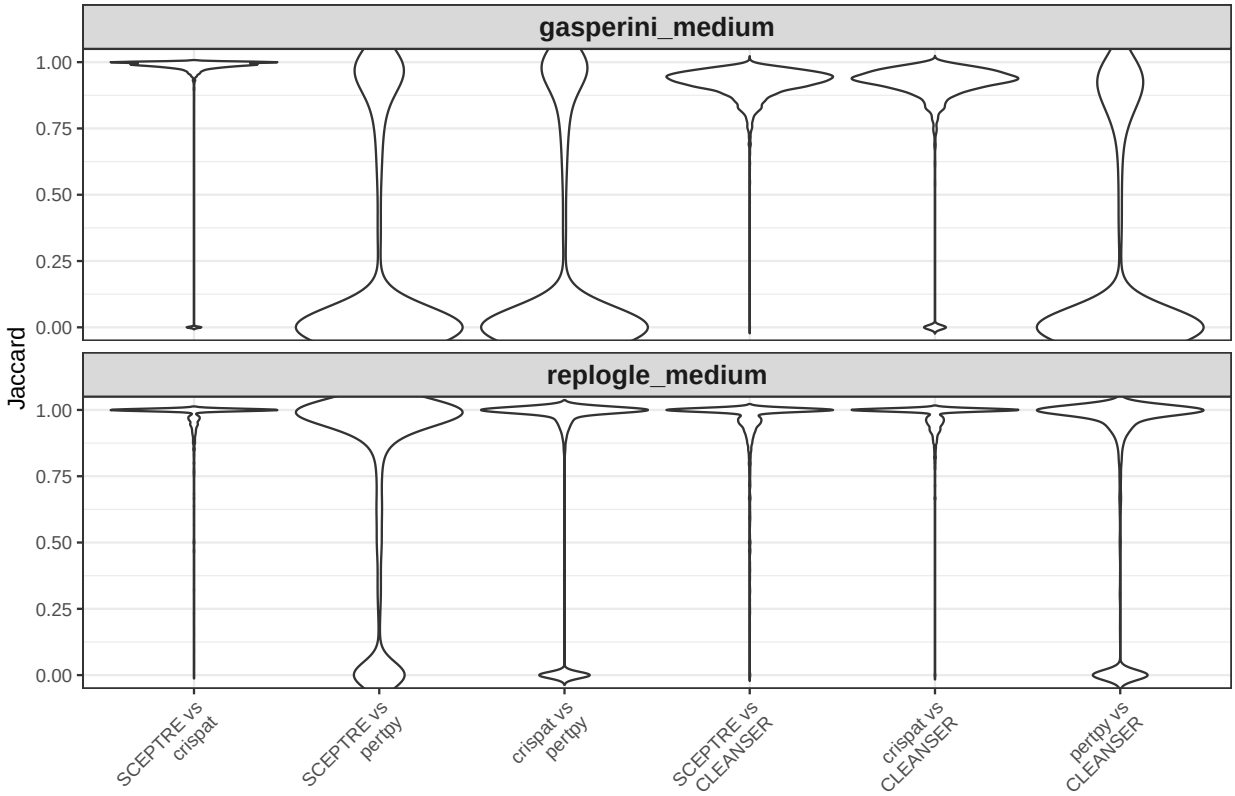
Table 2: Simulations: Computational Performance

Method	Runtime (min)	Peak memory (GB)
0.5% Pert (10 guides)		
SCEPTRE	**0.21**	**0.31**
crispat	8.70	1.10
CLEANSER	13.80	2.90
pertpy	7.77	3.90
1.5% Pert (25 guides)		
SCEPTRE	**0.37**	**0.31**
crispat	10.53	1.10
CLEANSER	50.18	8.60
pertpy	12.98	5.40

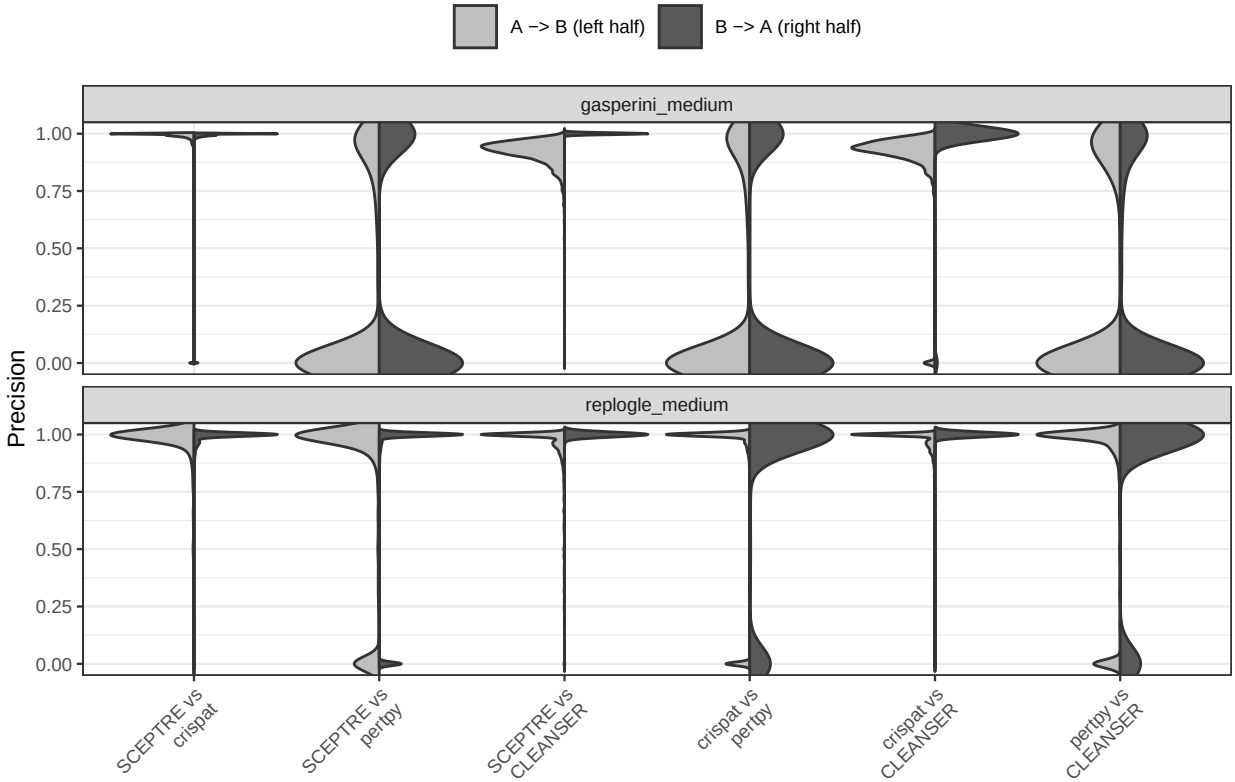
2b. Results on real data

For this section, all methods were run on downsampled versions of the data. This is due to computation time for `pertpy` and memory for `CLEANSER`. These datasets, denoted `replogle-rd7_medium` and `gasperini_medium`, each contain the first 2,500 guides and the first 50,000 cells as they appear in the data.

Jaccard similarity by method pair



Precision by method pair (split by direction)



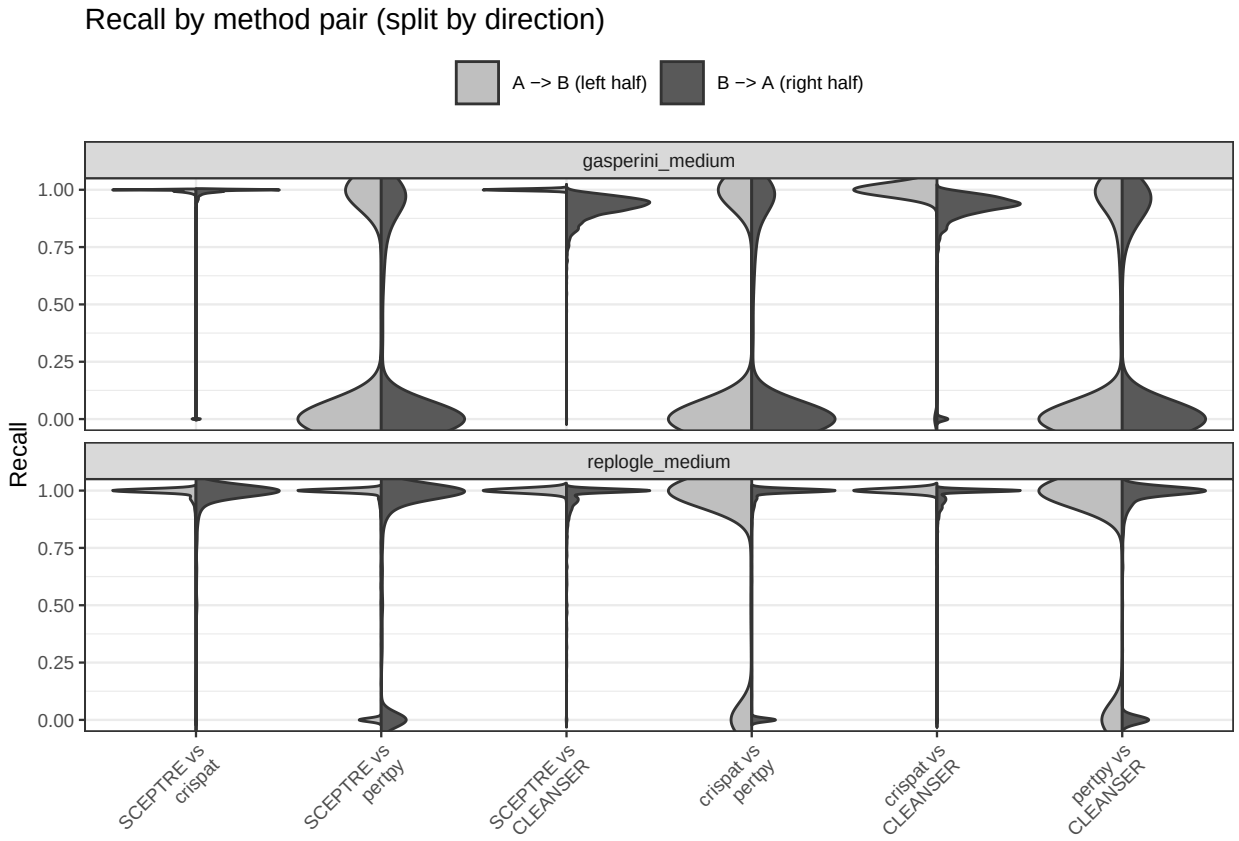
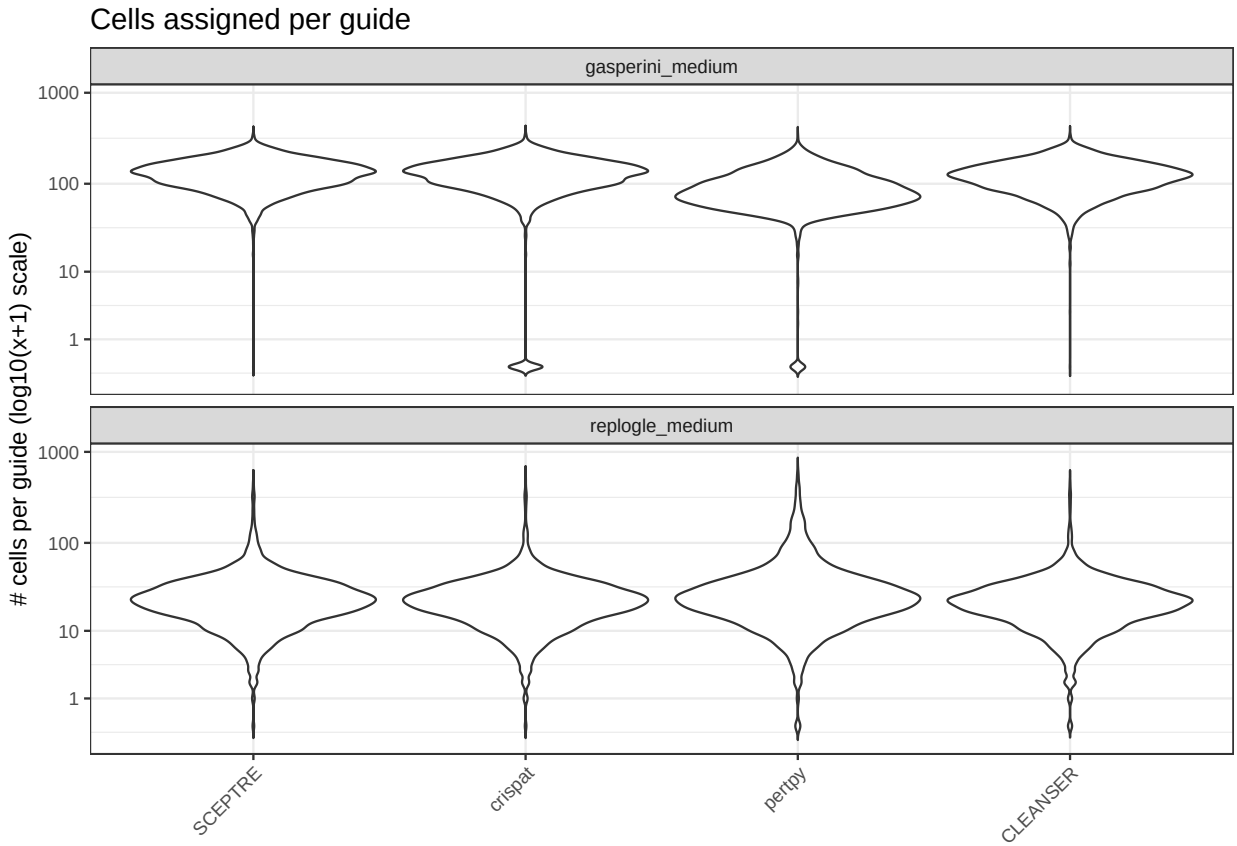


Table 3: Pairwise Jaccard similarity across guides (mean ± 95% CI).

Dataset	Method pair	Mean ± 95% CI
gasperini_medium	SCEPTRE vs crispat	0.960 ± 0.006
	SCEPTRE vs pertpy	0.264 ± 0.016
	SCEPTRE vs CLEANSER	0.919 ± 0.002
	crispat vs pertpy	0.253 ± 0.016
	crispat vs CLEANSER	0.899 ± 0.006
	pertpy vs CLEANSER	0.256 ± 0.016
replogle_medium	SCEPTRE vs crispat	0.953 ± 0.005
	SCEPTRE vs pertpy	0.756 ± 0.015
	SCEPTRE vs CLEANSER	0.940 ± 0.006
	crispat vs pertpy	0.785 ± 0.015
	crispat vs CLEANSER	0.972 ± 0.003
	pertpy vs CLEANSER	0.775 ± 0.015


Cells assigned per guide
gasperini_medium
replogle_medium
cells per guide (log10(x+1) scale)
1000
100
10
1
SCEPTRE
crispat
pertpy
CLEANSER

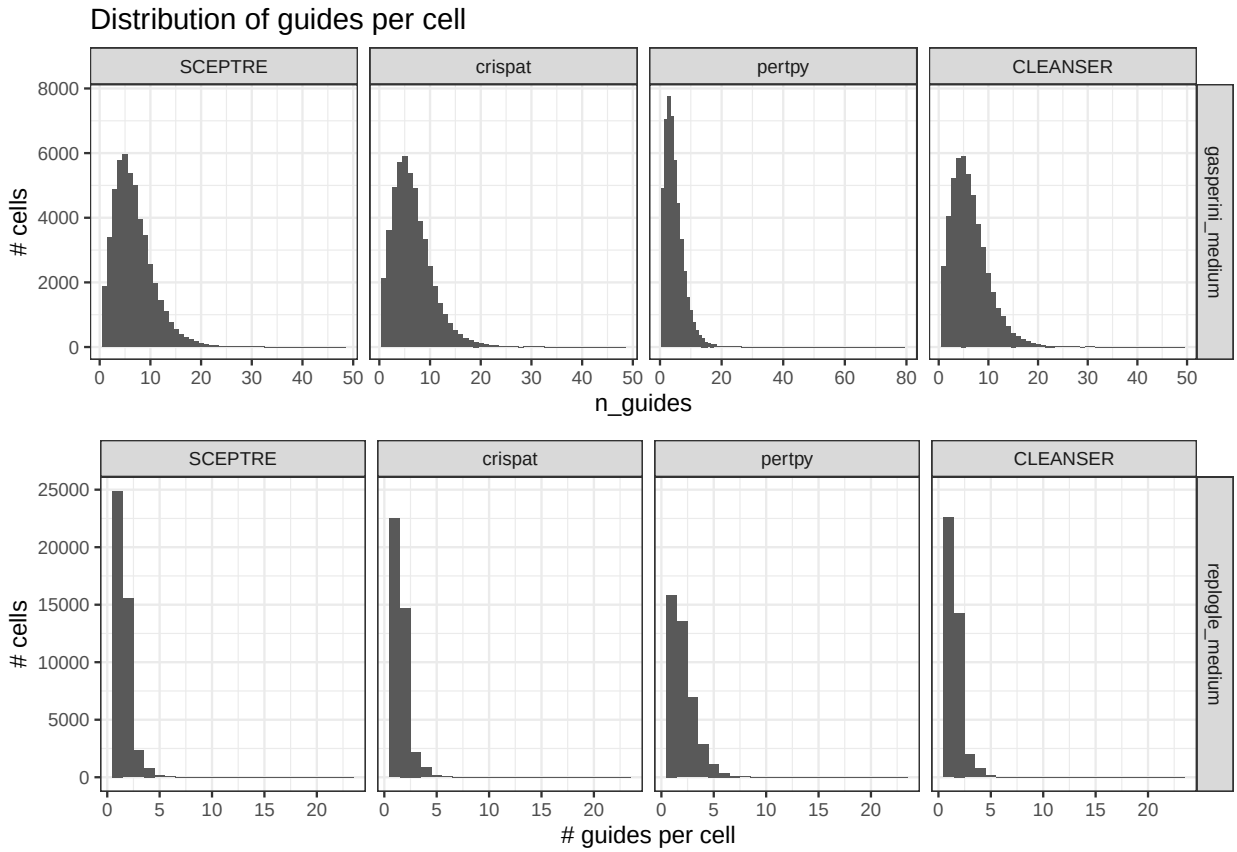
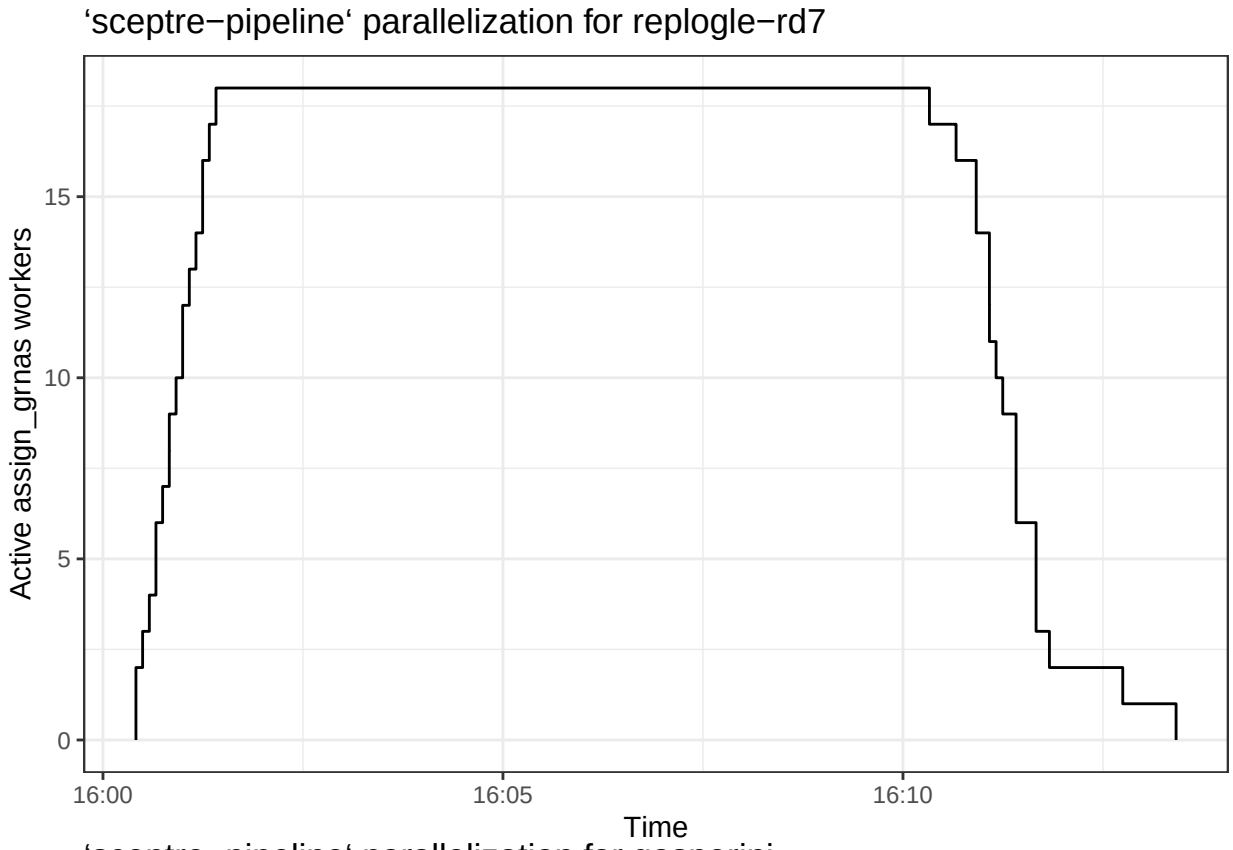

Distribution of guides per cell
SCEPTRE
crispat
pertpy
CLEANSER
gasperini_medium
replogle_medium
cells
n_guides
guides per cell

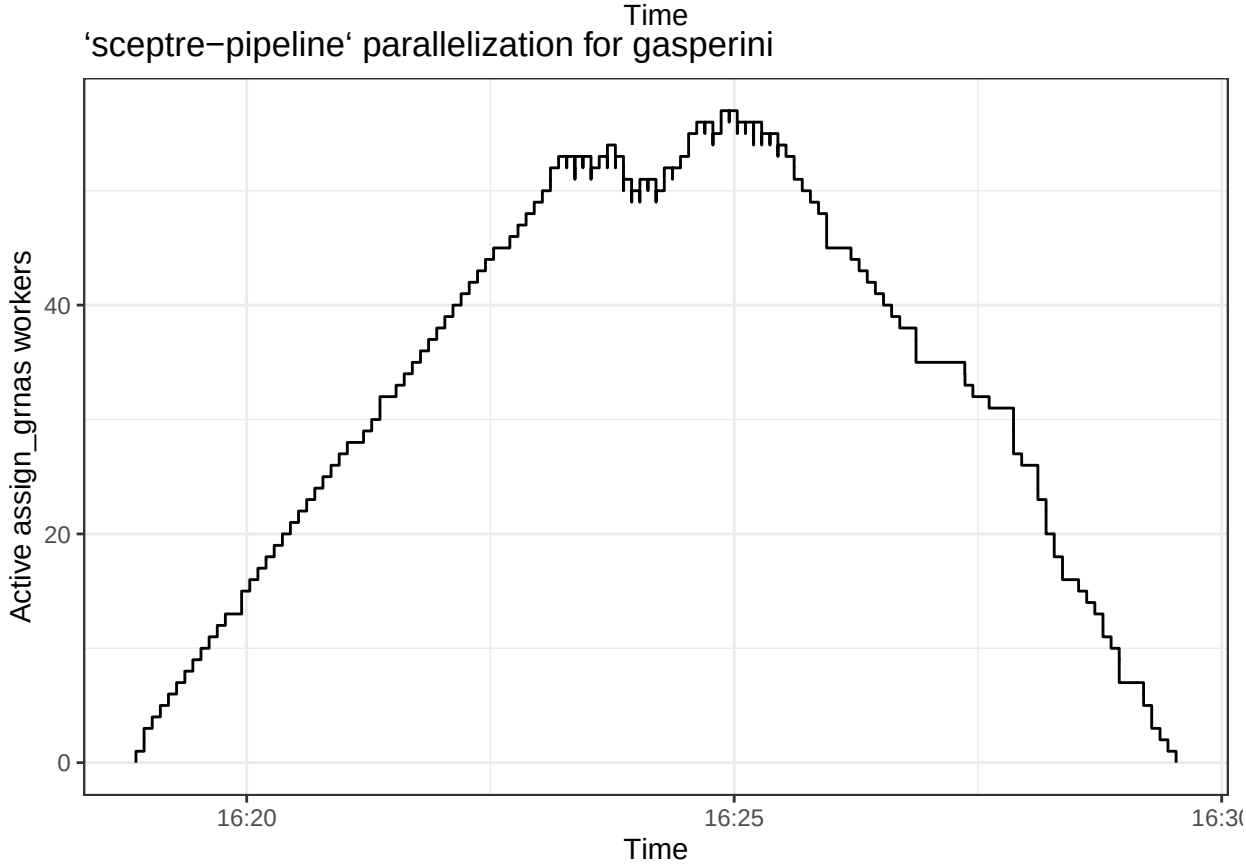
Table 4: Average assignment sizes: cells per gRNA and gRNAs per cell.

Dataset	Method	Avg cells per gRNA	Avg gRNAs per cell
gasperini_medium	SCEPTRE	131.9	6.68
	crispat	129.1	6.57
	pertpy	90.6	4.73
	CLEANSER	122.5	6.29
replogle_medium	SCEPTRE	27.0	1.54
	crispat	25.3	1.56
	pertpy	33.9	2.07
	CLEANSER	24.5	1.54

Table 5: Real data: Computational Performance

Method	Runtime (min)	Peak memory (GB)
gasperini_medium		
SCEPTRE	**10.63**	**0.29**
crispat	103.03	1.40
CLEANSER	217.55	54.10
pertpy	533.00	7.70
replogle-rd7_medium		
SCEPTRE	**12.43**	**0.30**
crispat	107.92	1.40
CLEANSER	211.62	49.60
pertpy	660.48	8.40


‘sceptre−pipeline‘ parallelization for replogle−rd7
Active assign_grnas workers
0
5
10
15
16:00
16:05
16:10
Time


‘sceptre−pipeline‘ parallelization for gasperini
Active assign_grnas workers
0
20
40
16:20
16:25
Time

TODO:

- array jobs to make these more overlapping?
- add datasets: barnyard? Fancier simulation?
- run on full datasets instead of medium? rename medium as "downsampled"? maybe do random downsample instead of 1:k?
- add methods: geomux?
- add assessments: held-out or resampling based? sensitivity to downsampling? F1 vs Jaccard?